02244 Logic for Security
Security Protocols
Channels and Composition

Sebastian Mödersheim

March 2, 2026

Challenge from Previous Module

Consider the following protocol:

```
A->B:  A,NA
B->s:  A,B,NA,NB,{|A,NA,NB|}sk(B,s)
s->A:  {|B,KAB,NA,NB|}sk(A,s),{|A,KAB|}sk(B,s)
A->B:  {|A,KAB|}sk(B,s),{|NB|}KAB
```

- Can you find a type-flaw attack against this protocol?
  Hint: ignore A and s and consider just one honest b in role B.

- Can you suggest formats for this protocol so that it becomes type-flaw resistant?

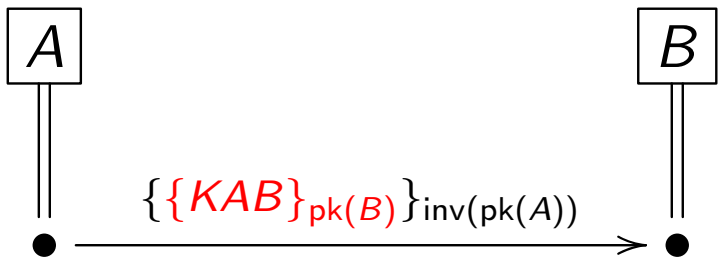
Challenge from Previous Module

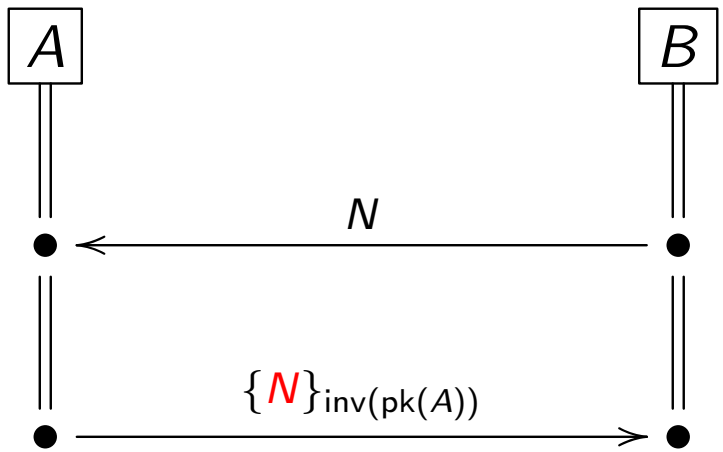
Consider the following protocol:

```
A->B:  A,NA
B->s:  A,B,NA,NB,{|A,NA,NB|}sk(B,s)
s->A:  {|B,KAB,NA,NB|}sk(A,s),{|A,KAB|}sk(B,s)
A->B:  {|A,KAB|}sk(B,s),{|NB|}KAB
```

- Can you find a type-flaw attack against this protocol?
  Hint: ignore A and s and consider just one honest b in role B.
  - ★ You can of course type the spec into OFMC, but let's see if we can find the attack using the hint and the lazy intruder technique.
- Can you suggest formats for this protocol so that it becomes type-flaw resistant?

Recall: Mind the Environment

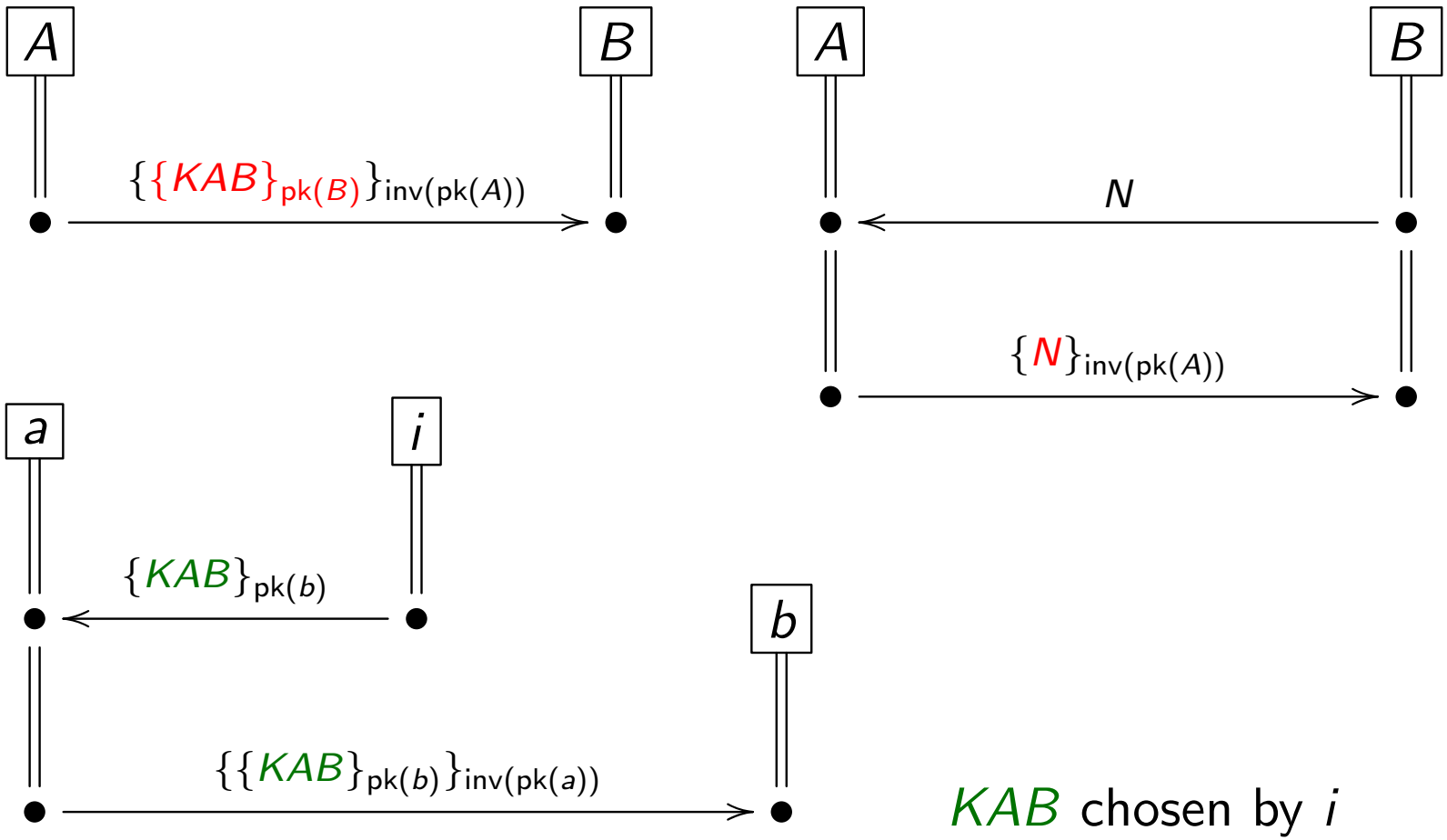




Terrible protocol design:

- while each of these protocols is (somewhat) secure in isolation
- together they break because the signed messages don't say what they mean...

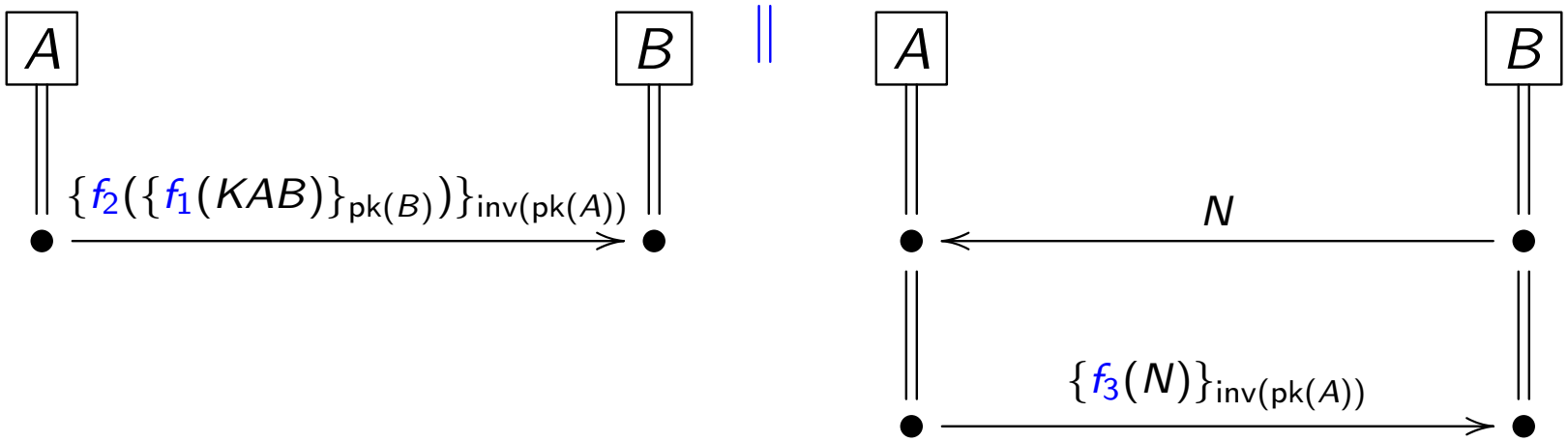
Recall: Mind the Environment



KAB chosen by *i*

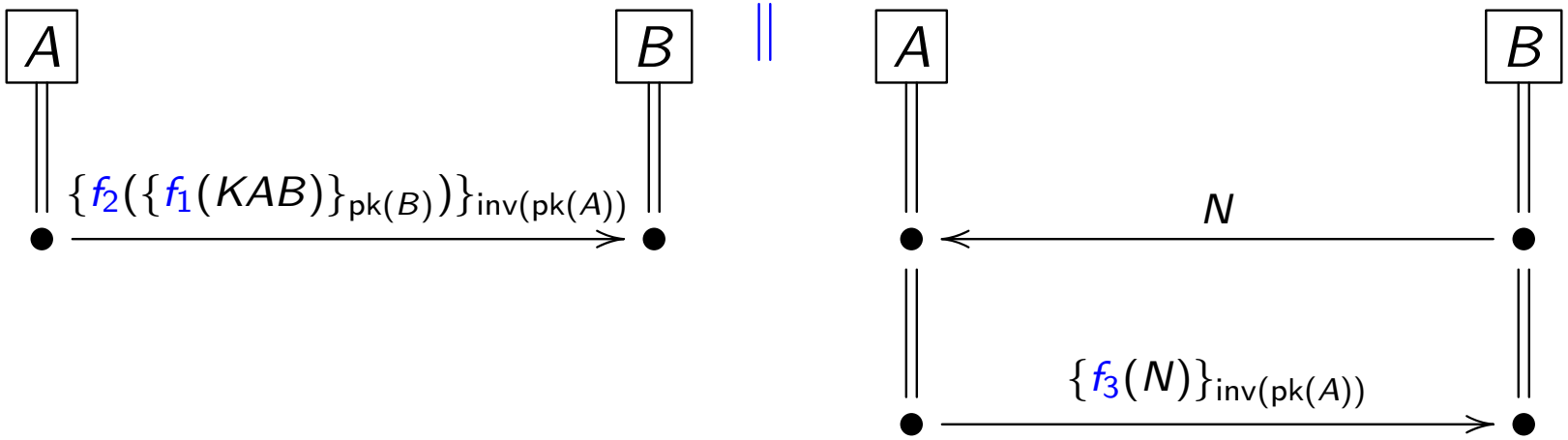
Attack/Confusion: *a* runs right protocol, *b* runs left protocol.
This is called a type flaw attack.

Parallel Composition



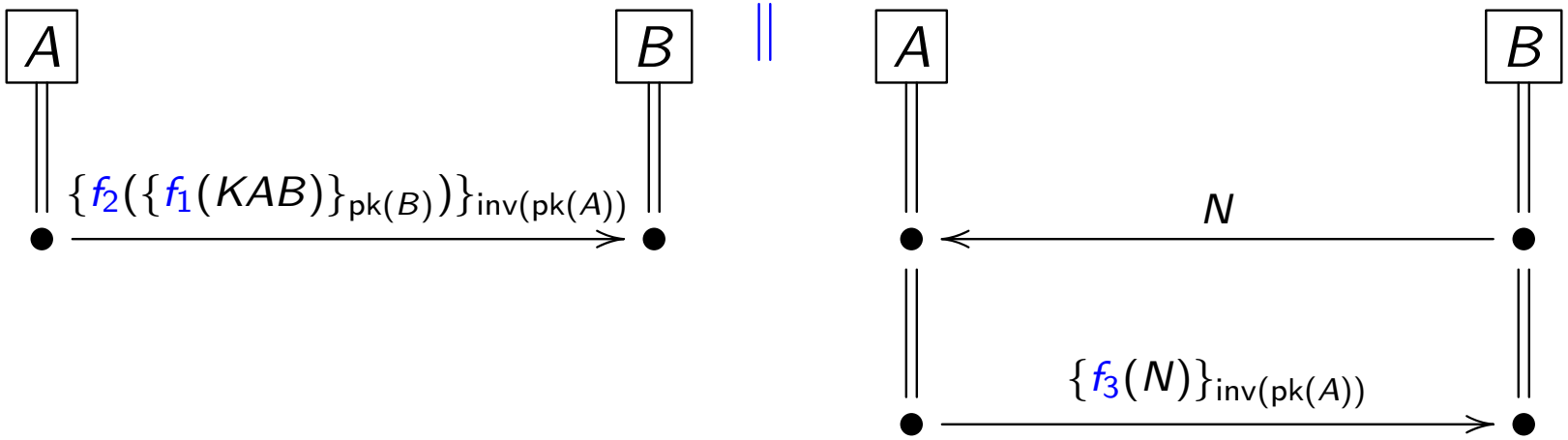
- When the two protocols are disjoint formats, the attack is not possible anymore.

Parallel Composition



- When the two protocols are disjoint formats, the attack is not possible anymore.
- Goal: Parallel Compositionality
  - ★ Verify: each component protocol is secure in isolation.
  - ★ Ensure some syntactic conditions like disjoint formats
  - ★ Then their parallel composition is also secure.

Parallel Composition

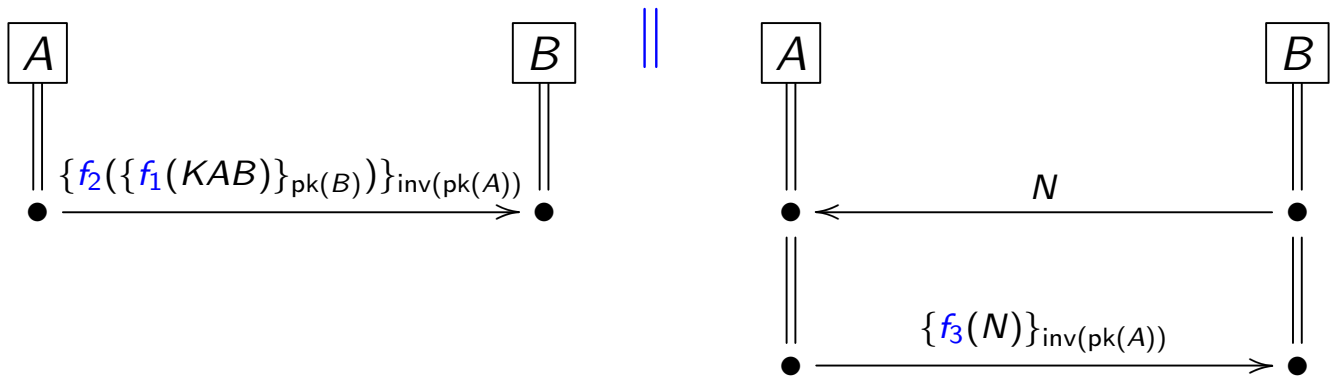


- When the two protocols are disjoint formats, the attack is not possible anymore.
- Goal: Parallel Compositionality
  - ★ Verify: each component protocol is secure in isolation.
  - ★ Ensure some syntactic conditions like disjoint formats
  - ★ Then their parallel composition is also secure.
- Relevant: there are tons of protocols on the Internet
  - ★ Verifying them all at once is too complex
  - ★ Every new addition would require verification to start from zero

Parallel Composition Result [Hess et al.'18]

Definition (Parallel Composability)

- Given type-flaw resistant protocols P_1 and P_2
- There is a special set of shared secrets (e.g. $\mathsf{inv}(\mathsf{pk}(a))$).
- Some of these secrets are explicitly declassified (e.g. $\mathsf{pk}(a), \mathsf{pk}(i), \mathsf{inv}(\mathsf{pk}(i))$).
- Every well-typed message part of P_1 and P_2 is
  - ★ either a shared secret (classified or declassified)
  - ★ or unique to one of the protocols
- Neither protocol (in isolation) ever leaks a classified secret.



Parallel Composition Result [Hess et al.'18]

Theorem (Parallel Composability)

If P_1 and P_2 are parallel composable and both secure in isolation, then also $P_1 \parallel P_2$ is secure.

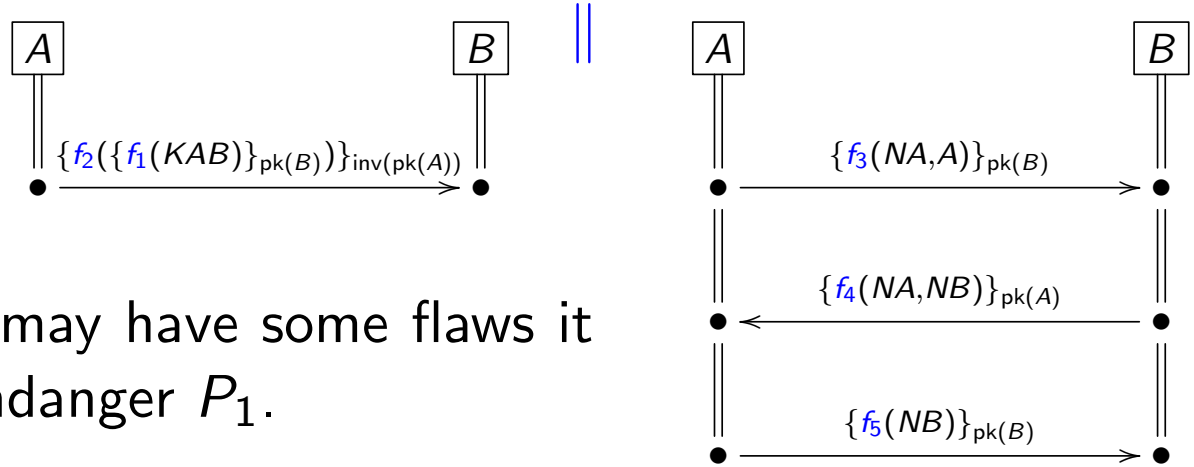
Parallel Composition Result [Hess et al.'18]

Theorem (Parallel Composability)

If P_1 and P_2 are parallel composable and both secure in isolation, then also $P_1 \parallel P_2$ is secure.

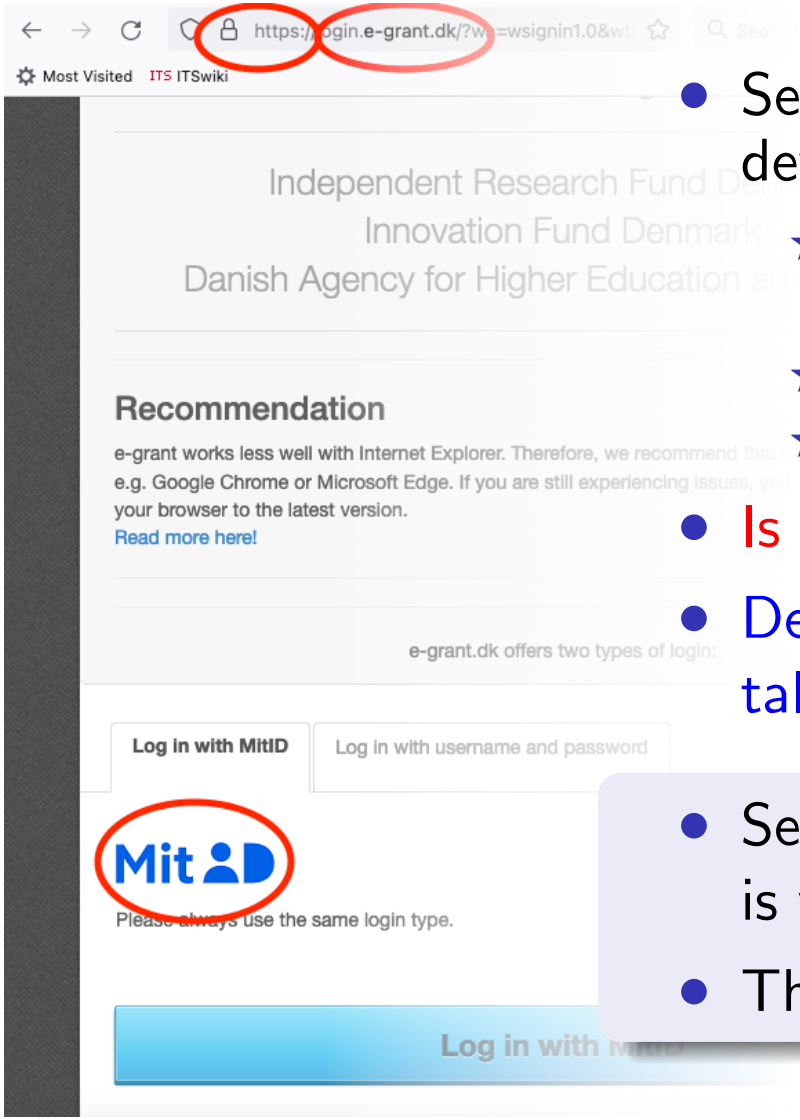
Theorem (Parallel Composability–Important Variant)

If P_1 and P_2 are parallel composable and P_1 is secure in isolation, then $P_1 \parallel P_2$ does not break any goals of P_1.



Even if P_2 may have some flaws it does not endanger P_1.

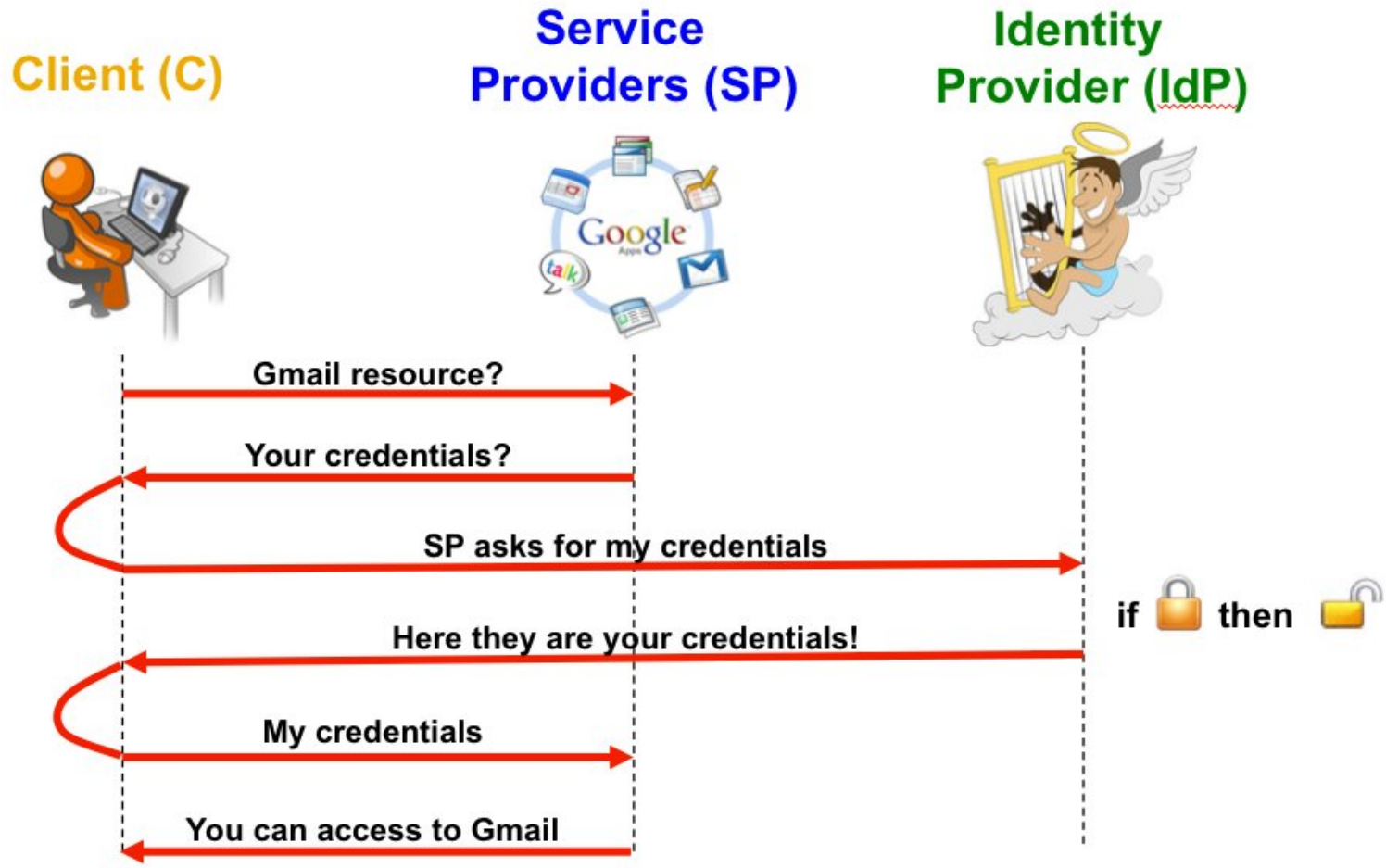
Vertical Composition: Example and Motivation



- Several components from independent developers:
  - ★ TLS (https): secure channels, authenticate servers
  - ★ MitID: authenticate user
  - ★ Payload application: e-grant system
- Is this secure?
- Developers cannot—and should not—all talk to each other!

- Security of components like TLS and MitID is well-understood.
- Their composition is not.

Example: Single Sign-On (SSO) with web-browser



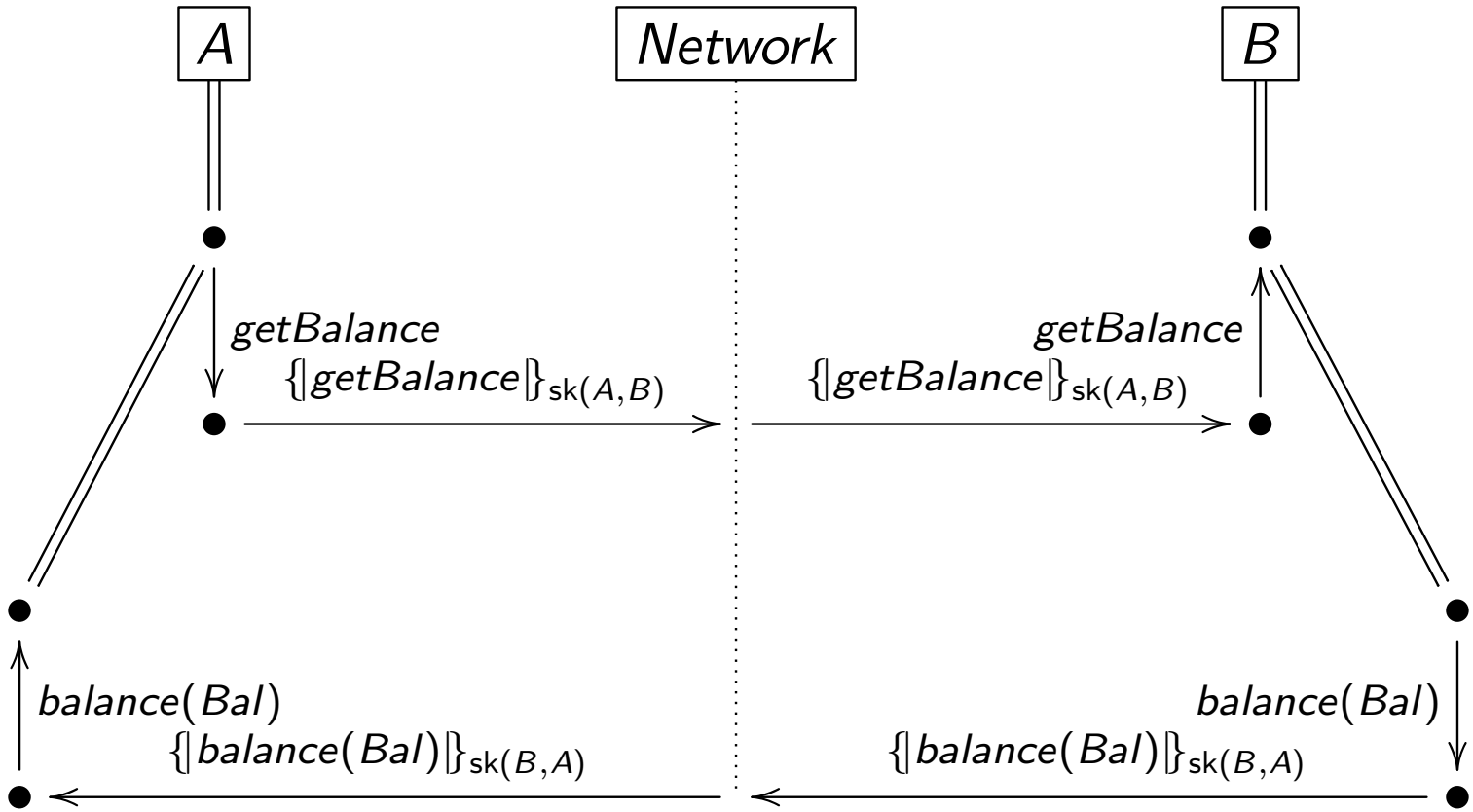
Leaving abstract two components here:

- The channels are running over TLS
- The client must in some way authenticate to the IdP

Google's SSO

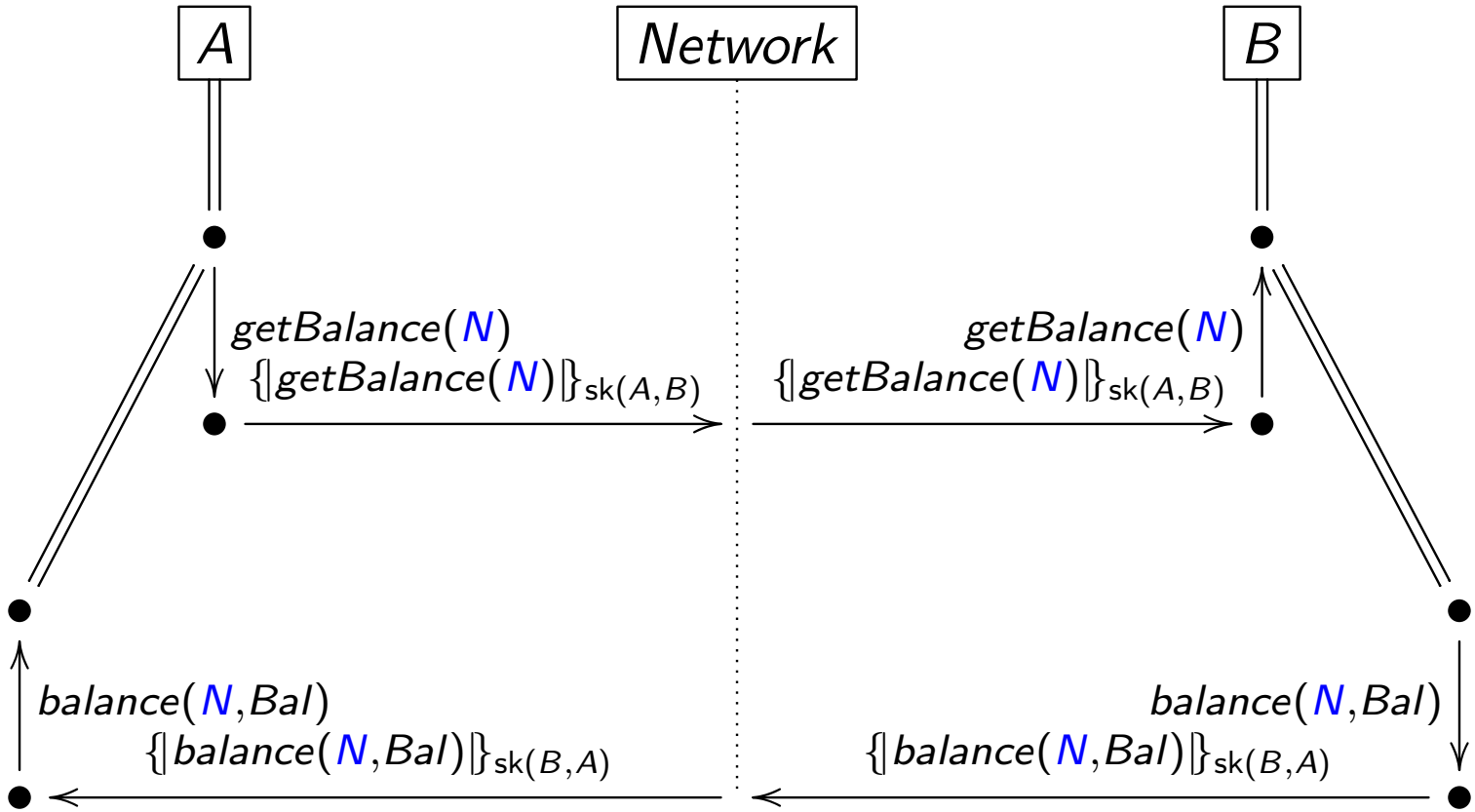
```
Knowledge: C: C,idp,SP,pk(idp);
           idp: C,idp,pk(idp),inv(pk(idp));
           SP: idp,SP,pk(idp)
Actions:
[C] *->* SP  : C,SP,URI
 SP *->* [C] : C,idp,SP,URI

 C  *->* idp : C,idp,SP,URI
idp *->* C   : {C,idp}inv(pk(idp)),URI

[C] *->* SP  : {C,idp}inv(pk(idp)),URI
 SP *->* [C] : Data
Goals:
SP authenticates C on URI
C authenticates SP on Data
Data secret between SP,C
```

Layering



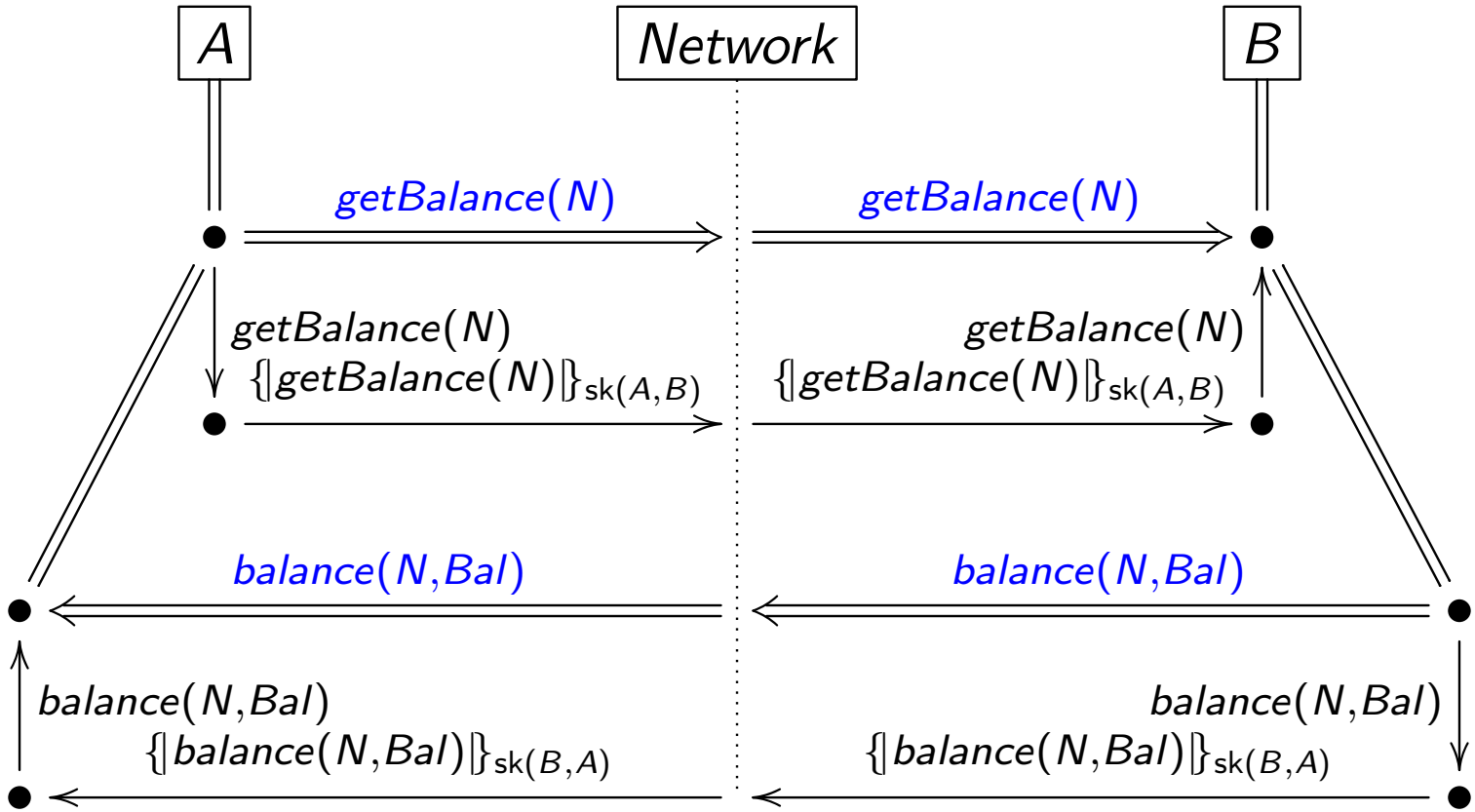
- Simple banking application: A asks for her account balance
- Simple channel: just encrypt with $\mathrm{sk}(A, B)$ or $\mathrm{sk}(B, A)$ (depending on direction)

Layering



Let's protect also against replay! (It is not done by this channel.)

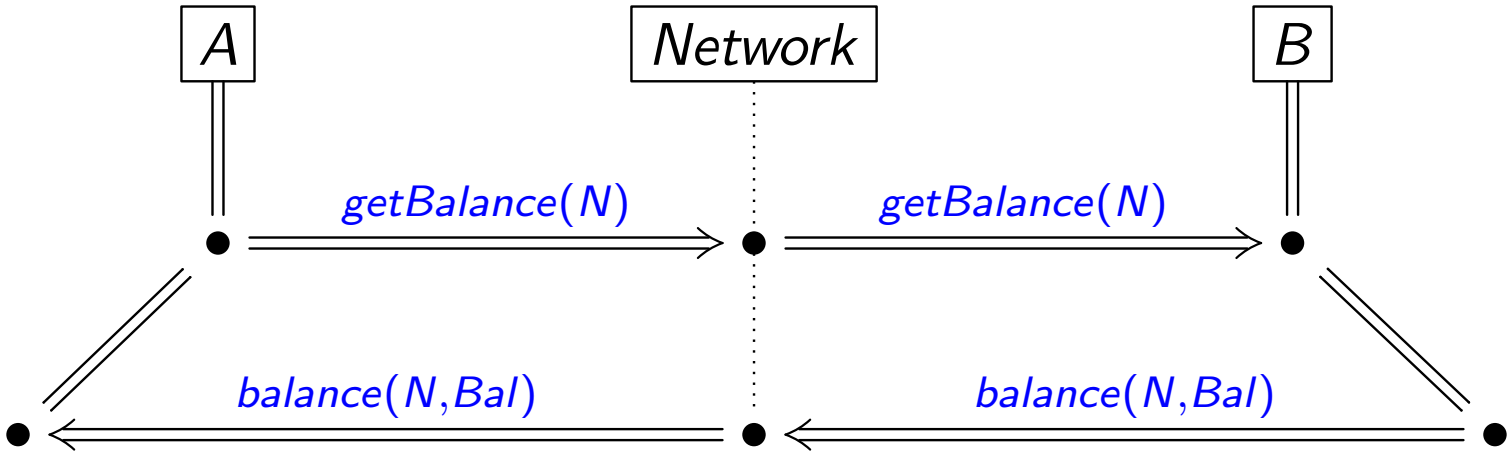
Layering



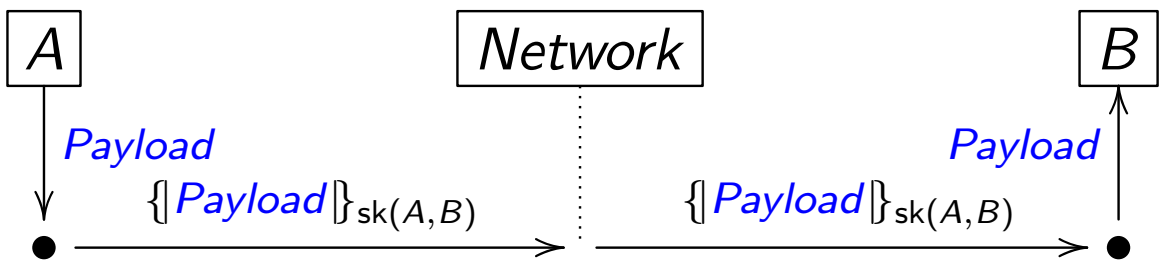
The channel is almost like a secure line between A and B.

Layering

Split between an application:

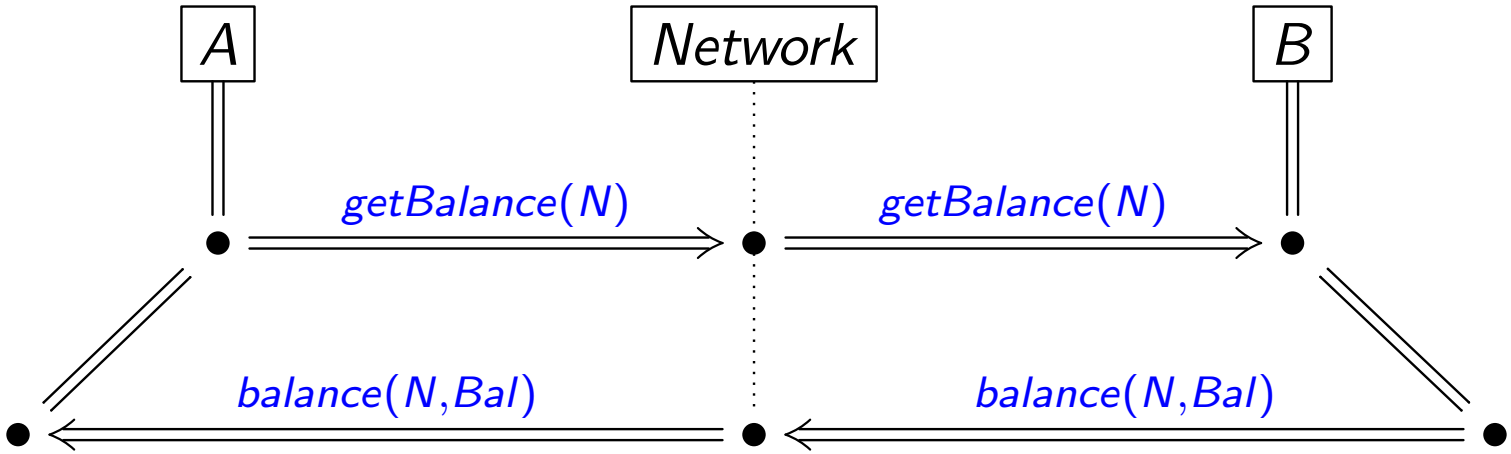


and a channel:

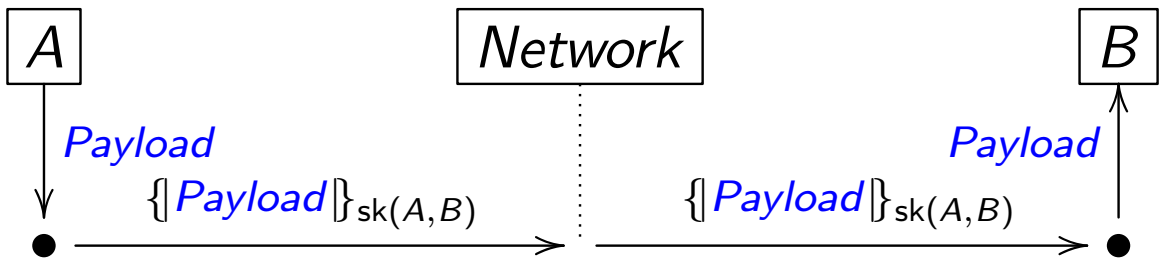


Layering

Split between an application:



and a channel:



Idea: we can replace the channel with something more complicated (like TLS), but we can verify the application with the simple channel.

Notation

Notation due to [Maurer and Schmidt]:

- $A \bullet\!\!\rightarrow B$ for an authentic channel from A to B.
  - ★ B can be sure the message comes from A, but we do not guarantee confidentiality.
- $A \rightarrow\!\!\bullet\, B$ for a confidential channel from A to B.
  - ★ A can be sure that only B can read this message, but we do not guarantee authenticity.
- $A \bullet\!\!\rightarrow\!\!\bullet\, B$ for a secure channel from A to B
  - ★ Both authentic and confidential.

OFMC supports this notation (write $\star$ for $\bullet$):

Example (NSL)

$$\begin{array}{lcll} A & \rightarrow & B: & \{NA, A\}_{\mathsf{pk}(B)} \\ B & \rightarrow & A: & \{NA, NB, B\}_{\mathsf{pk}(A)} \\ A & \rightarrow & B: & \{NB\}_{\mathsf{pk}(B)} \end{array}$$

Notation

Notation due to [Maurer and Schmidt]:

- $A \bullet\!\!\rightarrow B$ for an authentic channel from A to B.
  - ★ B can be sure the message comes from A, but we do not guarantee confidentiality.
- $A \rightarrow\!\!\bullet\, B$ for a confidential channel from A to B.
  - ★ A can be sure that only B can read this message, but we do not guarantee authenticity.
- $A \bullet\!\!\rightarrow\!\!\bullet\, B$ for a secure channel from A to B
  - ★ Both authentic and confidential.

OFMC supports this notation (write $\star$ for $\bullet$):

Example (NSL)

$$\begin{array}{llll} A & \rightarrow\bullet & B: & NA, A \\ B & \rightarrow\bullet & A: & NA, NB, B \\ A & \rightarrow\bullet & B: & NB \end{array}$$

- using confidential channels instead of crypto.

Implementing Channels with Cryptography

- Assume every agent A has two key pairs:
  - ★ $\langle \mathsf{ck}(A), \mathsf{inv}(\mathsf{ck}(A)) \rangle$ for asymmetric encryption.
  - ★ $\langle \mathsf{ak}(A), \mathsf{inv}(\mathsf{ak}(A)) \rangle$ for digital signatures.
- Assume that every agent knows the public keys $\mathsf{ck}(A)$ and $\mathsf{ak}(A)$ of every other agent A.
- Encode channels by encryption and signing:

Definition

$$
\begin{array}{llllllll}
A & \bullet\!\!\rightarrow & B: & M & \text{for} & A & \rightarrow & B: \quad \{\mathsf{atag}, B, M\}_{\mathsf{inv}(\mathsf{ak}(A))} \\
A & \rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: \quad \{\mathsf{ctag}, M\}_{\mathsf{ck}(B)} \\
A & \bullet\!\!\rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: \quad \{\{\mathsf{stag}, B, M\}_{\mathsf{inv}(\mathsf{ak}(A))}\}_{\mathsf{ck}(B)}
\end{array}
$$

- atag, ctag, and stag are tags to distinguish the channel-encodings from other encryptions.

A Cryptographic Implementation

Definition

$$
\begin{array}{llllllll}
A & \bullet\!\!\rightarrow & B: & M & \text{for} & A & \rightarrow & B: & \{\mathsf{atag}, B, M\}_{\mathsf{inv}(\mathsf{ak}(A))} \\
A & \rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: & \{\mathsf{ctag}, M\}_{\mathsf{ck}(B)} \\
A & \bullet\!\!\rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: & \{\{\mathsf{stag}, B, M\}_{\mathsf{inv}(\mathsf{ak}(A))}\}_{\mathsf{ck}(B)}
\end{array}
$$

- This ensures the basic properties of channels:
  - ★ Only A can produce messages on the channel $A \bullet\!\!\rightarrow B$.
  - ★ Only B can read messages on the channel $A \rightarrow\!\!\bullet B$.
  - ★ Both restrictions on a secure channel.
- Note that the intruder can still intercept and replay messages!
- This model of channels can be used with all models and tools without extensions!

Authenticated Recipient

Definition

$$
\begin{array}{llllllll}
A & \bullet\!\!\rightarrow & B: & M & \text{for} & A & \rightarrow & B: & \{\text{atag}, B, M\}_{\text{inv}(\text{ak}(A))} \\
A & \rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: & \{\text{ctag}, M\}_{\text{ck}(B)} \\
A & \bullet\!\!\rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: & \{\{\text{stag}, B, M\}_{\text{inv}(\text{ak}(A))}\}_{\text{ck}(B)}
\end{array}
$$

Why is the recipient B contained in the signatures?

- Including the name B means to authenticate the intended recipient.

Channel Calculus

Channel Calculus Rule

- If we have an authentic channel $A \bullet\!\!\rightarrow B$
- Then we can achieve a confidential channel $B \rightarrow\!\!\bullet A$ in the opposite direction

Channel Calculus

Channel Calculus Rule

- If we have a confidential channel $A \rightarrow\!\bullet\, B$
- Then we can achieve an authentic channel $B \bullet\!\rightarrow A$ in the opposite direction

Channel Calculus

Channel Calculus Rule

- If we have a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$
- Then we can achieve a secure channel $B \bullet\!\!\rightarrow\!\!\bullet\, A$ in the opposite direction.

Channel Calculus Rule

- If we have an authenticated channel $A \bullet\!\!\rightarrow B$
- and a confidential channel $A \rightarrow\!\!\bullet\, B$.
- Then we can achieve a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$.

- Challenge: Try to prove these two rules yourself.
- It is also obvious we can "downgrade" from secure to authentic or to confidential.
- This is all we can achieve without further assumptions.
- Challenge: Let s be an honest server. Suppose we have channels $s \bullet\!\!\rightarrow\!\!\bullet\, A$ and $s \bullet\!\!\rightarrow\!\!\bullet\, B$, can we achieve $A \bullet\!\!\rightarrow\!\!\bullet\, B$? Why does s have to be honest for that?

TLS handshake (simplified)

$$
\begin{array}{rl}
A \to B: & A, B, exp(g, X), cert_A \\
B \to A: & exp(g, Y), \\
\boxed{\text{let } \mathrm{k1} = \mathrm{clientK}(\mathrm{exp}(\mathrm{exp}(\mathrm{g},\mathrm{X}),\mathrm{Y}))} & \\
\boxed{\text{let } \mathrm{k2} = \mathrm{serverK}(\mathrm{exp}(\mathrm{exp}(\mathrm{g},\mathrm{X}),\mathrm{Y}))} & \\
 & \{\!|cert_B|\!\}_{k_2}, \\
 & \{\!|\{h(\text{all previous messages})\}_{\mathrm{inv}(\mathrm{pk}(B))}|\!\}_{k_2} \\
A \to B & \{\!|\{h(\text{all previous messages})\}_{\mathrm{inv}(\mathrm{pk}(A))}|\!\}_{k_1}
\end{array}
$$

$cert_A = \{A, pk(A), \ldots\}_{\mathrm{inv}(\mathrm{pk}(s))}$ is a public key certificate, and h, *clientK*, *serverK* are hash/key-derivation functions.

Consider also the transmission of a payload with the created keys:

$$
\begin{array}{rrrl}
 & & A \to B: & \{\!|data, DATA_A|\!\}_{k_1} \\
 & & B \to A: & \{\!|data, DATA_B|\!\}_{k_2} \\
\hline
Goal: & A \bullet\!\!\to\!\!\bullet\, B: & & DATA_A \\
 & B \bullet\!\!\to\!\!\bullet\, A: & & DATA_B
\end{array}
$$

Recall: Channel Calculus

Channel Calculus Rule

- If we have a confidential channel $A \rightarrow\bullet\ B$
- Then we can achieve an authentic channel $B \bullet\rightarrow A$ in the opposite direction

$A \quad \rightarrow\bullet \quad B : \quad K$
$B \quad \rightarrow \quad A : \quad \{|Payload|\}_K$

- Actually this is also a pseudonymous channel!
- Notation suggested by [M. & Viganò] and supported by OFMC:

$$B \bullet\rightarrow\bullet [A] \quad \text{and} \quad [A] \bullet\rightarrow\bullet B$$

A and B have a secure channel except B can't be sure who A is.

- TLS without client authentication establishes this channel.
- Login using TLS is then: $[A] \bullet\rightarrow\bullet B : login, A, password(A, B)$
- Which finally achieves $A \bullet\rightarrow\bullet B$

Cryptographic Model of Secure Pseudonymous Channels

OFMC implements pseudonymous channels as follows:

Definition

$$
\begin{array}{llllllll}
[A:P] & \bullet\!\!\rightarrow & B: & M & \text{for} & A & \rightarrow & B: & \{\text{atag}, B, M\}_{\text{inv}(P)} \\
A & \rightarrow\!\!\bullet & [B:P]: & M & \text{for} & A & \rightarrow & B: & \{\text{ctag}, M\}_{P} \\
[A:P] & \bullet\!\!\rightarrow\!\!\bullet & B: & M & \text{for} & A & \rightarrow & B: & \{\{\text{stag}, B, M\}_{\text{inv}(P)}\}_{\text{ck}(B)} \\
A & \bullet\!\!\rightarrow\!\!\bullet & [B:P]: & M & \text{for} & A & \rightarrow & B: & \{\{\text{stag}, P, M\}_{\text{inv}(\text{ak}(A))}\}_{P}
\end{array}
$$

where we explicitly annotate the pseudonym/public-key P being used. By default, we have a fresh public-key P for every protocol session and agent.

Vertical Composition Result

Sébastien Gondron and Sebastian Mödersheim. *Vertical Composition and Sound Payload Abstraction for Stateful Protocols.* CSF 2021:

- Definition of channel protocols
  - ★ This entails an ideal functionality similar to $A{\rightarrow}\bullet\, B$.
  - ★ Uses an abstract notion of payload messages
- Definition of application protocols
- Application and Channel interact with each other through buffers $\mathit{inbox}(A, B)$ and $\mathit{outbox}(A, B)$ that contain payload messages.
- Requirements on their message formats:
  - ★ No overlap, and requirements of parallel composability
- The application is secure with the ideal functionality of the channel.
- The channel indeed implements the ideal functionality with all abstract payloads that are
  - ★ are either known or unknown to the intruder
  - ★ are either novel or a repetition
- Then also the vertical composition is secure.

Challenges

As a little exercises

- Consider the challenges of the channel calculus.
- Consider the banking example of page 17 (actual pdf-pagenumber).
  - ★ Extend the banking application by a transfer message where A can specify an amount of money and a recipient C.
  - ★ Protect the transfer against replay (without changing the channel).
  - ★ How could one implement a channel that has replay protection built in?
  - ★ Suppose A later claims she never made the transfer. Can B prove to an honest third party *judy* ("without reasonable doubt") that A indeed *did* make the transfer?

Relevant Research Papers

- Ueli Maurer, Pierre Schmid: *A Calculus for Security Bootstrapping in Distrib. Systems*. J. of Comp. Sec. 4(1) 1996.
- Joshua Guttman and F. Javier Thayer: *Protocol independence through disjoint encryption*. CSFW 2000.
- Alessandro Armando et al.: *Formal analysis of SAML 2.0 web browser single sign-on*. FMSE 2008.
- Sebastian Mödersheim and Luca Viganò. *Secure Pseudonymous Channels*. ESORICS 2009.
- Véronique Cortier and Stéphanie Delaune. *Safely composing security protocols*. Formal Methods Syst. Des. 34(1), 2009.
- Joshua Guttman: *Cryptographic Protocol Composition via the Authentication Tests*. FOSSACS 2009. *Secure Composition of PKIs with Public Key Protocols*. CSF 2017.
- Andreas Hess, Sebastian Mödersheim and Achim Brucker. *Stateful Protocol Composition*. ESORICS 2018.
- Andreas Hess, Sebastian Mödersheim, and Achim Brucker. *Stateful Protocol Composition in Isabelle/HOL.* In ACM Trans. on Privacy and Security, 2023.
- Jan Camenisch et al.: *iUC: Flexible Universal Composability Made Simple*. ASIACRYPT 2019.
- Sébastien Gondron and Sebastian Mödersheim. *Vertical Composition and Sound Payload Abstraction for Stateful Protocols.* CSF 2021. http://imm.dtu.dk/ samo/AbstractPayload.pdf